Domain Expert Formalizers for Automated Theorem Proving

Connor Olson

Introduction

LLMs can generate proofs for mathematical statements, but utility is limited since hallucinations are common and verification is tedious. Proof assistants like Lean 4 can greatly reduce this burden by formally verifying the correctness of LLM generated proofs, now a popular approach to automated theorem proving.

State of the art automatic theorem proving systems generate formal, verified proofs for arbitrary informal statements by combining feedback from the Lean compiler, LLMs doing informal and formal reasoning, and a number of other tools. Benchmarking or using such systems can be extremely expensive, with the current leading system expending up to $1,012 on compute per proof at inference time (based on current SoTA Aleph Prover [5], as listed on PutnamBench [9]). One mechanism to reduce this burden in two effective systems, HILBERT [10] and DSP+ [1], is to use separate LLMs to perform reasoning and formalization. A first heavyweight "Reasoner" writes an informal proof (e.g. in LaTeX), and then a second (less expensive) model attempts to translate, or formalize, that proof into Lean 4. In addition, each of these systems ultimately rely on such a model that takes context procured or generated during an agentic loop, and attempts to provide a proof of the theorem, subgoal, or lemma.

These "Prover" models whose sole task is to write the formal proof are heavily post-trained on Lean 4 syntax, and informal-formal pairs of proofs, since (in principle) they do not need to perform much high level reasoning and are simply filling in details and translating a proof into Lean 4. However, it happens often that these prover systems must make many attempts in order to formalize a proof, so the prover LLM may be invoked hundreds or thousands of times to complete a single proof in full from end to end. By increasing confidence in the result of attempts of the prover, or increasing the performance of the prover, the overall cost and performance of such an proving system can be greatly improved.

Since the prover models are already highly specialized, one direction for improving their performance could be to further specialize them by fine tuning them for specific domains. As an obvious example, geometry problems tend to use a different set of features in Lean 4 since support is limited and the existing library is smaller, so specialized LLMs specifically for geometry have been able to outperform generalists [2] [8]: it is possible that separating the prover model step into multiple models managed by a routing network, one for geometry, and one for non-geometry, both fine-tuned on those particular domains, could increase the overall performance and decrease the cost of a prover system.

Approach

We investigate the efficacy of domain-specific prover models by fine-tuning a generalist LLM separately on geometry proofs and non-geometry proofs, and evaluating the pass@1 performance on static, pre-extracted informal-formal pairings with oracle routing by problem type, as compared to the base model and a model fine-tuned on all the data used to fine-tune both specialists.

Each model attempts to provide a formal proof for a given formal subgoal, given the full informal proof as context. Models are fine-tuned using QLoRA [3] with samples from the FormL4 [6] dataset and Herald [4] dataset, preprocessed to a simpler format, which provides intermediate proof states and tactic-level annotations. The testing set also contains Mathlib geometry proofs reverse-annotated with informal reasoning via a frontier LLM, as shown to be effective in multiple other works [4] [6] [11]. To isolate domain specialization as the independent variable, the non-geometry training subset is strictly undersampled to match the final geometry dataset size.

We benchmark the three following configurations:

1. Qwen3.5-9B [7] off the shelf.
2. Two specialist QLoRA fine-tuned Qwen3.5-9B models, one only on geometry and one on everything else, with an oracle router based on problem type to the appropriate model.
3. Qwen3.5-9B fine tuned on the combined aggregate dataset of both the specialist models above.

Results

The first benchmark is a `pass@1` evaluation for each model on the theorem level held out test set.

Table 1: `pass@1` Performance

Category	Base	Fine-Tuned Generalist	Routed Specialists
Geometry	3/98 (3.1%)	15/98 (15.3%)	17/98 (17.3%)
Non-Geometry	12/197 (6.1%)	19/197 (9.6%)	17/197 (8.6%)
Overall	15/295 (5.1%)	34/295 (11.5%)	34/295 (11.5%)

For `pass@1`, the oracle routed specialists successfully completed more geometry proofs, but less non-geometry proofs, resulting in identical performance with the generalist fine-tuned on the combination of the specialist's data. The oracle routed specialists successfully completed more geometry proofs, but less non-geometry proofs, resulting in the same overall performance with the generalist fine-tuned on the combination of the specialist's data.

Table 2: `pass@2` Performance

Category	Base	Fine-Tuned Generalist	Routed Specialists
Geometry	3/98 (3.1%)	16/98 (16.3%)	18/98 (18.4%)
Non-Geometry	12/197 (6.1%)	24/197 (12.2%)	23/197 (11.7%)
Overall	15/295 (5.1%)	40/295 (13.6%)	41/295 (13.9%)

The `pass@2` performance further separates the fine tuned models from the base model, and in this case the routed specialists actually outperform the generalist model overall, but still solve less non-geometry problems. In this benchmark, if the model fails to write a proof on it's first attempt, it receives the compiler feedback from Lean and it's previous attempt in a subsequent one.

One example of a geometry proof successfully completed by the geometry specialist that the generalist model could not complete is provided below:

Theorem to complete:

```
theorem EuclideanGeometry.sin_oangle_right_mul_dist_of_oangle_eq_pi_div_two {p₁ p₂ p₃ : P}
    (h : ∡ p₁ p₂ p₃ = ↑(π / 2)) : Real.Angle.sin (∡ p₂ p₃ p₁) * dist p₁ p₃ = dist p₁ p₂ := by
  sorry
```

Generalist model response (failing):

```
<think>
The theorem states that if the oriented angle `∡ p₁ p₂ p₃` is π/2, then `Real.Angle.sin
(∡ p₂ p₃ p₁) * dist p₁ p₃ = dist p₁ p₂`. We can rotate the points cyclically without
affecting the validity of the oriented angle condition. First, we establish that the sign
of the oriented angle `∡ p₂ p₃ p₁` is 1 by rotating the hypothesis `h` and checking that
```

```
π/2 has a positive sign. Using `oangle_eq_angle_of_sign_eq_one`, we equate the oriented
angle to its unoriented counterpart. We then convert the sine of the oriented angle to the
sine of the real angle using `Real.Angle.sin_real`. Finally, we apply the standard
unoriented right-angle lemma `sin_angle_mul_dist_of_angle_eq_pi_div_two` after reordering
the points to complete the proof.
</think>
have hs : (∡ p₂ p₃ p₁).sign = 1 := by
rw [← oangle_rotate_sign, h, Real.Angle.sign_coe_pi_div_two]
  rw [oangle_eq_angle_of_sign_eq_one hs, Real.Angle.sin_real,
    sin_angle_mul_dist_of_angle_eq_pi_div_two]
```

Specialist model response (passing):

```
<think>
The goal is to prove `Real.Angle.sin (∡ p₂ p₃ p₁) * dist p₁ p₃ = dist p₁ p₂` given that
the oriented angle `∡ p₁ p₂ p₃` is a right angle. Just like its unoriented counterpart,
this identity represents the length of the opposite side in an oriented right-angled
triangle. We can transform the oriented sine term into the unoriented sine by evaluating
the sign of the angle `∡ p₂ p₃ p₁`. Rotating a right angle configuration preserves its
orientation, so the sign is `1`, which allows us to replace the oriented angle with the
unoriented angle `∠ p₂ p₃ p₁` and equate their sines. This reduces the problem to the
standard unoriented right-angle lemma `sin_oangle_right_mul_dist_of_oangle_eq_pi_div_two`,
which matches the geometric configuration after the angle is unoriented.
</think>
have hs : (∡ p₂ p₃ p₁).sign = 1 := by rw [oangle_rotate_sign, h, Real.Angle.sign_coe_pi_div_two]
  rw [oangle_eq_angle_of_sign_eq_one hs, Real.Angle.sin_coe,
    sin_angle_mul_dist_of_angle_eq_pi_div_two (angle_eq_pi_div_two_of_oangle_eq_pi_div_two h)]
```

Looking closely, we notice that the generalist model is capable of the same high level reasoning as the specialist, but ultimately fails to apply the `rw` tactic appropriately in this case, leaving out a needed argument for the oriented angle, and making a small syntactic mistake with a directional tactic application.

Conclusions

The positive conclusion we can draw is that using a geometry specific model for geometry oriented proofs is definitely better, and proving systems would benefit from routing geometry oriented subproblems to a prover model that is specifically trained for geometry.

However, there is no conclusive evidence that any further decomposition of models into specialists would benefit a proving system. As the performance of the generalist exceeds the oracle routed specialists in other domains, it does not seem like a promising direction for future work. It is possible that the limited number of proofs and the small model sizes contributed to this result, but it seems unlikely from these results alone that scaling would flip the conclusion in the opposite direction, when usually scaling, generality, and performance all go together.

Limitations

This project had a budget of $0.00 and exclusively used free trial credits from cloud or model providers for training, evaluation, and generating synthetic data. For this reason, only a single small model was evaluated. It is unclear if results have any chance of generalizing to larger model classes.

Training examples were limited by VRAM. The prompt (see Appendix B) maintains generality by including a lot of information about the proof, so extremely long proofs could not be trained on or evaluated. Generally speaking, the point of a prover system is to shortcut the agentic system when possible and complete the proof, but this metric is not contained in these results since any large proof could not be tested. Only a sufficiently decomposed theorem would be representable by the used dataset, which required a context length of ~ 4096 tokens at the very most, although examples were trimmed below this to prevent OOM.

Existing SOTA systems do adopt architectures where the prover/formalizer step is relevant, but only due to economic or other inference constraints. For pure performance or in more general systems with different architectures, it is unclear if this result holds any significance.

The primary results of this paper may simply be due to data scarcity. For example, it is not out of the question that differences in performance on geometry problems for autoformalization in Lean is simply due to the lack of existing library support. It is possible that all issues addressed by this result would be better addressed by contributing to Mathlib.

The prompt used to generate chain of thought traces was short sighted. See the prompt in appendix C, and the resulting traces on Hugging Face: the traces are likely not the best representation of how a model should actually "think" about writing a proof, as many examples simply recite the proof strategy instead of reasoning about Lean or the goals created by an application of a tactic.

References

[1] Chenrui Cao, Liangcheng Song, Zenan Li, Xinyi Le, Xian Zhang, Hui Xue, and Fan Yang. Reviving dsp for advanced theorem proving in the era of reasoning models, 2025.

[2] Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving, 2025.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.

[4] Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated lean 4 dataset, 2025.

[5] Logical Intelligence. Aleph prover: State-of-the-art formal theorem prover, 2026. Accessed: 2026-04-29.

[6] Jianqiao Lu, Yingjia Wan, Zhengying Liu, Yinya Huang, Jing Xiong, Chengwu Liu, Jianhao Shen, Hui Jin, Jipeng Zhang, Haiming Wang, Zhicheng Yang, Jing Tang, and Zhijiang Guo. Process-driven autoformalization in lean 4, 2024.

[7] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.

[8] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

[9] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024.

[10] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning, 2026.

[11] Huajian Xin et al. Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search. *arXiv preprint arXiv:2408.08152*, 2024.

Appendix A : Data

The derivative dataset used in this project is available on Hugging Face. The format is as follows:

Table 3: Dataset Column Descriptions

Column	Description
item_id	A UUIDv4 string acting as the absolute unique identifier for this specific extracted subgoal.
source_dataset	The categorical origin of the data (e.g., `herald`, `forml4`, `mathlib`).
original_id	A composite grouping key (e.g., `module::theorem_name`) pointing to the original source. Used to enforce theorem-level train/test splits.
informal_context	The full informal theorem statement and the entirety of the corresponding human proof sketch.
formal_context_with_sorry	The complete, standalone Lean code for the theorem, containing exactly one `sorry` that the model is intended to fill in. If this string is passed to Lean, it must compile with only a "declaration uses sorry" warning.
formal_history	The formal Lean code written from the theorem declaration up to the current subgoal, providing structural context.
tactic_state	The exact local context, hypotheses, and goal active at the precise location of the `sorry`.
formal_completion	A complete Lean 4 tactic block (e.g., a `by` block or multiple tactics) that **closes** the subgoal currently completed by `sorry` (leaving no remaining goals for this branch).
is_geometry	Boolean flag indicating whether this problem belongs to the geometry domain.

Each upstream dataset is lacking some of the information provided by these data items, or only contains a full formal proof without intermediate states. To keep evaluation simple, We extract intermediate states from LeanDojo and stored them statically. For original data items without informal proofs, We use a frontier LLM to generate the informal proof from the formal proof.

All data was processed to ensure it compiles in Lean v4.11.0. This means it was compiled in Lean v4.11.0, and if any errors were thrown, the example was not used in training or testing. Data was initially categorized as Geometry using a structural rule that strictly identifies `Mathlib.Geometry.*` module paths, effectively excluding unrelated modules such as `Mathlib.AlgebraicGeometry.*`. This strategy reflects the purpose of the division, which is based on Lean 4 support for this subject area.

Appendix B : Fine Tuning

Note that the upstream Herald dataset contains no chain of thought in its informal proofs, while Qwen3.5 uses chain of thought. Fine tuning without the `<think>` blocks may be lovingly referred to as "lobotomizing" the model. As I did not have resources to use a frontier LLM to generate artificial `<think>` blocks for every data item, the CoT dataset is substantially smaller than the lobotomized dataset ($\sim$ 200 CoT rows compared to $> 50,000$ lobotomy rows). To address this, each model was fine-tuned in two sequential phases.

In the first phase (Lobotomy), the model was fine-tuned using a system prompt explicitly instructing it not to reason extensively, mapped to the lobotomized data with empty `<think>` blocks. The second phase

(CoT) continued training the resulting adapter on the smaller CoT dataset using a system prompt requiring careful thought.

To prevent catastrophic forgetting of the fast-response behavior during the CoT phase, a 50% replay strategy was employed: Phase 2 batches were mixed to contain equal parts CoT examples (presented with the CoT system prompt) and Phase 1 lobotomy examples (presented with the lobotomy system prompt). Theorem IDs were globally partitioned such that no theorem ever appeared as both a lobotomy and a CoT target.

All training was executed on a single L4 GPU (24 GB VRAM). The base model, Qwen3.5-9B, utilizes a hybrid Gated DeltaNet linear-attention architecture rather than a standard full-attention transformer, permitting a 4096 token context window without exceeding memory budgets. We utilized 4-bit NF4 quantization, BF16 gradients, and the `paged_adamw_8bit` optimizer via QLoRA. Learning rates were scaled down across phases (2×10^{-4} in Phase 1 to 5×10^{-5} in Phase 2), paired with an extended warmup ratio in Phase 2 (15%) to prevent forgetting when resetting the AdamW optimizer state for the CoT data distribution.

The resulting adapters are available on Hugging Face:

- https://huggingface.co/connorolson/qwen35-9b-lean4-generalist-lora
- https://huggingface.co/connorolson/qwen35-9b-lean4-nongeometry-lora
- https://huggingface.co/connorolson/qwen35-9b-lean4-geometry-lora

Appendix C : Prompts

The model was tuned with examples formatted in ChatML. As the prover is designated with closing a particular subgoal in a proof, the ChatML prompt specifies this and asks the prover to fill in the `sorry` such that the top level theorem is completed. Depending on the prover system, this may or may not reflect the use case of such a model. It is possible that subgoals would be reformatted as their own top level theorems with the proof context, or that data would be passed in a different way. To maintain generality, the prompt simply includes all the information that is relevant.

To accommodate the fine tuning phases, one prompt specifies not to think and uses the majority of data with empty `<think>` blocks, and another specifies to think and is used with data containing artificial CoT traces.

Lobotomized prompt:

```
<|im_start|>system
    You are an expert Lean 4 theorem prover. Your task is to complete the formal proof
    by providing the tactic or sequence of tactics required to close all goals from the
    `sorry` position. Respond directly with the tactic(s) without extended reasoning.
<|im_end|>
<|im_start|>user
    INFORMAL CONTEXT:
    {informal_context}

    CURRENT TACTIC STATE:
    {tactic_state}

    LEAN CODE:
    {formal_context_with_sorry}
<|im_end|>
<|im_start|>assistant
    ```
    Target completion format (from `train_lobotomy.jsonl`):
    ```
    <think>
```

```
    </think>
    {formal_completion}
<|im_end|>
```

Chain of thought prompt:

```
<|im_start|>system
    You are an expert Lean 4 theorem prover. Your task is to complete the formal
    proof by providing the tactic or sequence of tactics required to close all
    goals from the `sorry` position. Think carefully through the proof strategy
    before providing your answer.
<|im_end|>
<|im_start|>user
    INFORMAL CONTEXT:
    {informal_context}

    CURRENT TACTIC STATE:
    {tactic_state}

    LEAN CODE:
    {formal_context_with_sorry}
<|im_end|>
<|im_start|>assistant
    ```
    Target completion format (from `train_CoT.jsonl`):
    ```
    <think>
    {synthetic_trace}
    </think>
    {formal_completion}
<|im_end|>
```

The prompt used to generate the CoT traces was:

```
You are writing synthetic chain-of-thought text for Lean 4 proof training data.
Return strict JSONL output only.
Each line must be exactly one JSON object with exactly these two fields:
{"row_index": <integer>, "CoT_trace": "<string>"}

Requirements:
- Output JSONL only. No Markdown. No code fences. No surrounding prose.
- Preserve each row_index exactly as given.
- CoT_trace must contain only the reasoning to go inside the <think>...</think> block.
- Do not include the final Lean proof code in CoT_trace.
- Keep the reasoning concise, mathematically relevant, and faithful to the proof.
- If a row is unclear, still provide your best concise reasoning rather than skipping it.
```

An adjusted prompt was also used for `pass@2` evaluation on the second pass:

```
<|im_start|>system
  You are an expert Lean 4 theorem prover. A previous proof attempt failed to compile.
  Given the original theorem context, the previous attempt, and Lean's feedback,
  produce a corrected Lean proof completion that replaces the same `sorry`.
  Output only Lean code. Do not include explanations, Markdown, or code fences.
<|im_end|>
<|im_start|>user
```

```
  INFORMAL CONTEXT:
  {informal_context}

  CURRENT TACTIC STATE:
  {tactic_state}

  LEAN CODE WITH SORRY:
  {formal_context_with_sorry}

  PREVIOUS ATTEMPT:
  {previous_prediction}

  LEAN FEEDBACK:
  {lean_feedback}
<|im_end|>
<|im_start|>assistant
  {corrected_formal_completion}
<|im_end|>
```